

Name

`uusub` – Monitors `uucp` network.

Syntax

`uusub [ options ]`

Description

Uusub defines a *uucp* subnetwork and monitors the connection and traffic among the members of the subnetwork. The following options are available:

- `-a sys` Adds *sys* to the subnetwork.
- `-d sys` Deletes *sys* from the subnetwork.
- `-l` Reports the statistics on connections.
- `-r` Reports the statistics on traffic amount.
- `-f` Flushes the connection statistics.
- `-u hr` Gathers the traffic statistics over the past *hr* hours.
- `-c sys` Exercises the connection to the system *sys*. If *sys* is specified as **all**, then exercise the connection to all the systems in the subnetwork.

The connections report format is:

```
sys #call #ok time #dev #login #nack #other
```

where *sys* is the remote system name, *#call* is the number of times the local system tries to call *sys* since the last flush was done, *#ok* is the number of successful connections, *time* is the latest successful connect time, *#dev* is the number of unsuccessful connections because of no available device (e.g., ACU), *#login* is the number of unsuccessful connections because of login failure, *#nack* is the number of unsuccessful connections because of no response (e.g., line busy, system down), and *#other* is the number of unsuccessful connections because of other reasons.

The traffic statistics format is:

```
sfile sbyte rfile rbyte
```

where *sfile* is the number of files sent and *sbyte* is the number of bytes sent over the period of time indicated in the latest *uusub* command with the `-u hr` option. Similarly, *rfile* and *rbyte* are the numbers of files and bytes received.

The command:

```
uusub -c all -u 24
```

is typically started by *cron*(C) once a day.

Files

/usr/spool/uucp/SYSLOG	System log file
/usr/lib/uucp/L_sub	Connection statistics
/usr/lib/uucp/R_sub	Traffic statistics

See Also

cron(C), *uucp*(C), *uustat*(C)

Name

uuto, *uupick* – Public XENIX-to-XENIX file copy.

Syntax

uuto [options] source-files destination

uupick [-s system]

Description

Uuto sends *source-files* to *destination*. *Uuto* uses the *uucp*(C) facility to send files, while it allows the local system to control the file access. A source-file name is a path name on your machine. Destination has the form:

system!user

where *system* is taken from a list of system names that *uucp* knows about (see *uname*(C)). *Logname* is the login name of someone on the specified system.

Two options are available:

- p Copy the source file into the spool directory before transmission.
- m Send mail to the sender when the copy is complete.

The files (or sub-trees if directories are specified) are sent to */usr/spool/uucppublic* on *system*. Specifically the files are sent to:

/usr/spool/uucppublic/receive/user/mysystem/files.

The destined recipient is notified by *mail*(C) of the arrival of files.

Uupick accepts or rejects the files transmitted to the user. Specifically, *uupick* searches PUBDIR for files destined for the user. For each entry (file or directory) found, the following message is printed on the standard output:

from system: [file file-name] [dir dirname] ?

Uupick then reads a line from the standard input to determine the disposition of the file:

- <new-line> Goes on to next entry.
- d Deletes the entry.
- m [dir] Moves the entry to named directory *dir* (current directory is default).

a [<i>dir</i>]	Same as m except moves all the files sent from <i>system</i> .
p	Prints the content of the file.
q	Stop.
EOT (control-d)	Same as q .
! <i>command</i>	Escape to the shell to do <i>command</i> .
*	Prints a command summary.

Upick invoked with the **--system** option will only search the PUBDIR for files sent from *system*.

Files

PUBDIR /usr/spool/uucppublic Public directory

See Also

mail(C), uuclean(C), uucp(C), uuname(C), uustat(C), uux(C)

Name

uux – Executes command on remote XENIX.

Syntax

uux [-] command-string

Description

Uux will gather zero or more files from various systems, execute a command on a specified system and send standard output to a file on a specified system.

The command-string is made up of one or more arguments that look like a shell command line, except that the command and filenames may be prefixed by:

system-name!

A null system-name is interpreted as the local system.

Filenames may be (1) a full pathname; (2) a pathname preceded by ~xxx; where xxx is a user ID on the specified system and is replaced by that user's login directory; or (3) anything else prefixed by the current directory.

The - option will cause the standard input to the *uux* command to be the standard input to the command-string.

For example, the command:

```
uux '!diff usg!/usr/dan/f1 pwba!/a4/dan/f1 > !fi.diff'
```

will get the **f1** files from the usg and pwba machines, execute a *diff* command and put the results in **f1.diff** in the local directory.

Any special shell characters such as <>| should be quoted either by quoting the entire command-string, or quoting the special characters as individual arguments.

Files

/usr/uucp/spool	Spool directory
/usr/uucp/*	Other data and programs

See Also

uucp(C)

Notes

Only the first command of a shell pipeline may have a system-name!. All other commands are executed on the system of the first command.

The shell metacharacter `*` will probably not perform as expected.

The shell tokens `<<` and `>>` are not implemented.

Uux does not notify if execution was denied on the remote machine.

Warning

An installation may, and for security reasons generally will, limit the list of commands executable on behalf of an incoming request from *uux*. Typically, a restricted site will permit little other than the receipt of mail via *uux*. The list of commands allowed is kept in the file */usr/lib/uucp/L.cmds* on the system that executes the program. You may add allowable commands by adding a line containing the name of the command (which must be present in */bin* or */usr/bin*) to the *L.cmds* file. Use a separate line for each command.

Name

vi – Invokes a screen-oriented display editor.

Syntax

vi [*-option ...*] [*command*] [*filename*]

edit [*-option ...*] [*command*] [*filename*]

vedit [*-option ...*] [*command*] [*filename*]

view [*-option ...*] [*command*] [*filename ...*]

Description

Vi offers a powerful set of text editing operations based on a set of mnemonic commands. Most commands are single keystrokes that perform simple editing functions. *Vi* displays a full screen “window” into the file you are editing. The contents of this window can be changed quickly and easily within *vi*. While editing, visual feedback is provided (the name *vi* itself is short for “visual”).

Vi and the line editor *ex* are the same editor: the names *vi* and *ex* identify a particular user interfaces rather than any underlying functional difference. The differences in user interfaces, however, are quite striking. *Ex* is a powerful line-oriented editor, similar to the editor *ed*. However, in both *ex* and *ed*, visual updating of the terminal screen is limited, and commands are entered on a command line. *Vi*, on the other hand, is a screen-oriented editor designed so that what you see on the screen corresponds exactly and immediately to the contents of the file you are editing. In the following discussion, *vi* commands are printed, initially, in boldface type; thereafter, the commands are printed in italics.

The *edit* command enters *ex* escape mode directly. *Vi*’s text window is not displayed; only the *ex* colon (:) prompt appears. You can enter any of the *ex* commands on the command line. See the following section, “Ex Commands,” for more information.

The *vedit* command is the same as the *vi* except that a message, “INPUT MODE”, displays at the bottom of the screen when the user is in insert mode.

The *view* command is the same as *vi* except that the read-only option (*-R*) is set automatically. The file cannot be changed with *view*.

Options available on the *vi* command line include:

-t Equivalent to an initial *tag* command; edits the file containing the tag and positions the editor at its definition.

- r** Used in recovering after an editor or system crash for retrieving the last saved version of the editing buffer. *Vi* periodically saves the changes made to a file during editing operations. Therefore, this option may not recover all of the changes made to the file before the crash, but it will recover the most recent version of the file that is available. Usually, all but the last two or three changes are recovered. If no file is specified, this option prints a list of saved files.
- l** Specific to editing LISP, this option sets the **showmatch** and **lisp** options.
- wn** Sets the default window size to *n*. Useful on dialups to start in small windows.
- R** Sets a read-only option so that files can be viewed but not edited.

The Editing Buffer

Vi performs no editing operations on the file that you name during invocation. Instead, it works on a copy of the file in an "editing buffer." The editor remembers the name of the file specified at invocation, so that it can later copy the editing buffer back to the named file. The contents of the named file are not affected until the changes are copied back to the original file. This allows editing of the buffer without immediately destroying the contents of the original file.

When you invoke *vi* with a single filename argument, the named file is copied to a temporary editing buffer. When the file is written out, the temporary file is written back to the named file.

Modes of Operation

Within *vi* there are three distinct modes of operation:

- | | |
|----------------|--|
| Command Mode | Within command mode signals from the keyboard are interpreted as editing commands. |
| Insert Mode | Insert mode can be entered by typing any of the <i>vi</i> insert, append, open, substitute, change, or replace commands. Once in insert mode, letters typed at the keyboard are inserted into the editing buffer. |
| Ex Escape Mode | The <i>vi</i> and <i>ex</i> editors are the same editor differing mainly in their user interface. In <i>vi</i> , commands are usually single keystrokes. In <i>ex</i> , commands are lines of text terminated by a RETURN. <i>Vi</i> has a special "escape" command that gives access to many of these line-oriented <i>ex</i> commands. To use the <i>ex</i> escape mode, type a colon (:). The colon is echoed on the status line as a prompt for the <i>ex</i> command. An executing command can be aborted by pressing |

INTERRUPT. Most file manipulation commands are executed in *ex* escape mode (for example, the commands to read in a file and to write out the editing buffer to a file).

Special Keys

There are several special keys in *vi*. The following keys are used to edit, delimit, or abort commands and command lines.

ESC	Used to return to <i>vi</i> command mode, cancel partially formed commands.
RETURN	Terminates <i>ex</i> commands when in <i>ex</i> escape mode. Also used to start a new line when in insert mode.
INTERRUPT	Often the same as the DELETE or RUBOUT key on many terminals. Generates an interrupt, telling the editor to stop what it is doing. Used to abort any command that is executing.
/	Used to specify a string to be searched for. The slash appears on the status line as a prompt for a search string. The question mark (?) works exactly like the slash key, except that it is used to search backward in a file instead of forward.
:	The colon is a prompt for an <i>ex</i> command. You can then type in any <i>ex</i> command, followed by an ESCAPE or RETURN and the given <i>ex</i> command is executed.

The following characters are special in insert mode:

BACKSPACE	Backs up the cursor one character on the current line. The last character typed before the BACKSPACE is removed from the input buffer, but remains displayed on the screen.
CTRL-U	Moves the cursor back to the first character of the insertion and restarts insertion.
CTRL-V	Removes the special significance of the next typed character. Use CTRL-V to insert control characters. LINEFEED and CTRL-J cannot be inserted in the text except as newline characters. CTRL-Q and CTRL-S are trapped by the operating system before they are interpreted by <i>vi</i> , so they too cannot be inserted as text.
CTRL-W	Moves the cursor back to the first character of the last inserted word.

- CTRL-T During an insertion, with the **autoindent** option set and at the beginning of the current line, typing this character will insert a **shiftwidth** white space.
- CTRL-@ If typed as the first character of an insertion, it is replaced with the last text inserted, and the insertion terminates. Only 128 characters are saved from the last insertion. If more than 128 characters were inserted, this command inserts no characters. A CTRL-@ cannot be part of a file, even if quoted.

Starting and Exiting Vi

To enter *vi*, type:

- | | |
|-----------------------|----------------------------------|
| <i>vi</i> | Edits empty editing buffer |
| <i>vi file</i> | Edits named file |
| <i>vi +123 file</i> | Goes to line 123 |
| <i>vi +45 file</i> | Goes to line 45 |
| <i>vi +/word file</i> | Finds first occurrence of “word” |
| <i>vi +/tty file</i> | Finds first occurrence of “tty” |

There are several ways to exit the editor:

- ZZ** The editing buffer is written to the file *only* if any changes were made.
- :x** The editing buffer is written to the file *only* if any changes were made.
- :q!** Cancels an editing session. The exclamation mark (!) tells *vi* to quit unconditionally. In this case, the editing buffer is not written out.

Vi Commands

Vi is a visual editor with a window on the file. What you see on the screen is *vi*'s notion of what the file contains. Commands do not cause any change to the screen until the complete command is typed. Most commands may take a preceding count that specifies repetition of the command. This count parameter is not given in the following command descriptions, but is implied unless overridden by some other prefix argument. When *vi* gets an improperly formatted command, it rings a bell.

Cursor Movement

The cursor movement keys allow you to move your cursor around in a file. Note in particular the direction keys (if available on your terminal), the H, J, K, and L cursor keys, and SPACEBAR, BACKSPACE, CTRL-N, and CTRL-P. These three sets of keys perform identical functions.

Forward Space – l, SPACEBAR, or RIGHT key

Syntax: **l**
 SPACEBAR
 RIGHT key

Function: Moves the cursor forward one character. If a *count* is given, move forward *count* characters. You cannot move past the end of the line.

BACKSPACE – h, BACKSPACE, or LEFT key

Syntax: **h**
 BACKSPACE
 LEFT key

Function: Moves cursor backward one character. If a *count* is given, moves backward *count* characters. Note that you cannot move past the beginning of the current line.

Next Line – +, RETURN, j, CTRL-N, and LF

Syntax: **+**
 RETURN

Function: Moves the cursor down to the beginning of the next line.

Syntax: **j**
 CTRL-N
 LF
 DOWN key

Function: Moves the cursor down one line, remaining in the same column. Note the difference between these commands and the preceding set of next line commands which move to the *beginning* of the next line.

Previous Line – k, CTRL-P, and UP key

Syntax: **k**
 CTRL-P
 UP key

Function: Moves the cursor up one line, remaining in the same column. If a *count* is given then the cursor is moved *count* lines.

Syntax: –

Function: Moves the cursor up to the beginning of the previous line. If a *count* is given then the cursor is moved up *count* lines.

Beginning of Line – 0 and ^

Syntax: ^
0

Function: Moves the cursor to the beginning of the current line. Note that 0 always moves the cursor to the first character of the current line. The caret (^) works somewhat differently: it moves to the first character on a line that is not a TAB or a SPACEBAR. This is useful when editing files that have a great deal of indentation, such as program texts.

End of Line – \$

Syntax: \$

Function: Moves the cursor to the end of the current line. Note that the cursor resides on top of the last character on the line. If a *count* is given, then the cursor is moved forward *count*–1 lines to the end of the line.

Goto Line – G

Syntax: [*linenumber*]G

Function: Moves the cursor to the beginning of the line specified by *linenumber*. If no *linenumber* is given, the cursor moves to the beginning of the *last* line in the file. To find the line number of the current line, use CTRL-G.

Column – |

Syntax: [*column*]

Function: Moves the cursor to the column in the current line given by *column*. If no *column* is given then the cursor is moved to the first column in the current line.

Word Forward – w and W

Syntax: w
W

Function: Moves the cursor forward to the beginning of the next word. The lowercase **w** command searches for a word defined as a string of alphanumeric characters separated by punctuation or whitespace (i.e., TAB, NEWLINE, or SPACEBAR characters). The uppercase **W** command searches for a word defined as a string of non-whitespace characters.

Back Word – b and B

Syntax: **b**
 B

Function: Moves the cursor backward to the beginning of a word. The lowercase **b** command searches backward for a word defined as a string of alphanumeric characters separated by punctuation or white space (i.e., TAB, NEWLINE, or SPACEBAR characters). The uppercase **B** command searches for a word defined as a string of non-whitespace characters. If the cursor is already within a word, then it moves backward to the beginning of that word.

End – e and E

Syntax: **e**
 E

Function: Moves the cursor to the end of a word. The lowercase **e** command moves the cursor to the last character of a word, where a word is defined as a string of alphanumeric characters separated by punctuation or white space (i.e., TAB, NEWLINE, or SPACEBAR characters). The uppercase **E** moves the cursor to the last character of a word where a word is defined as a string of non-whitespace characters. If the cursor is already within a word, then it moves to the end of that word.

Sentence – (and)

Syntax: (
)

Function: Moves the cursor to the beginning (left parenthesis) or end of a sentence (right parenthesis). A sentence is defined as a sequence of characters ending with a period (.), question mark (?), or exclamation mark (!), followed by either two spaces or a NEWLINE. A sentence begins on the first non-whitespace character following a preceding sentence. Sentences are also delimited by paragraph and section delimiters. See below.

Paragraph – { and }

Syntax: }
 {

Function: Moves the cursor to the beginning ({} or end (}) of a paragraph. A paragraph is defined with the *paragraphs* option. By default, paragraphs are delimited by the *nroff* macros .IP, .LP, .P, .QP, and .bp. Paragraphs also begin after empty lines.

Section – [[and]]

Syntax:]]
 [[

Function: Moves the cursor to the beginning ([[or end (]]) of a section. A section is defined with the *sections* option. By default, sections are delimited by the *nroff* macros .NH and .SH. Sections also start at FORMFEEDs (CTRL-L) and at lines beginning with a brace ({}).

Match Delimiter – %

Syntax: %

Function: Moves the cursor to a matching delimiter, where a delimiter is a parenthesis, a bracket, or a brace. This is useful when matching pairs of nested parentheses, brackets, and braces.

Home – H

Syntax: [*offset*]H

Function: Moves the cursor to upper left corner of screen. Use this command to quickly move to the top of the screen. If an *offset* is given, then the cursor is moved *offset*-1 number of lines from the top of the screen. Note that the “dH” command deletes all lines from the current line to the top line shown on the screen.

Middle Screen – M

Syntax: M

Function: Moves the cursor to the beginning of the screen’s middle line. Use this command to quickly move to the middle of the screen from either the top or the bottom. Note that the “dM” command deletes from the current line to the line specified by the *M* command.

Lower Screen – L

Syntax: [*offset*]L

Function: Moves the cursor to the lowest line on the screen. Use this command to move quickly to the bottom of the screen. If an *offset* is given, then the cursor is moved *offset*-1 number of lines from the bottom of the screen. Note that the “dL” command deletes all lines from the current line to the bottom line shown on the screen.

Previous Context – `` and ``

Syntax: ``
 `character`
 ``
 `character`

Function: Moves the cursor to previous context or to context marked with the *m* command. If the single quotation mark or back quotation mark is doubled, then the cursor is moved to previous context. If a single character is given after either quotation mark, then the cursor is moved to the location of the specified mark as defined by the *m* command. Previous context is the location in the file of the last “nonrelative” cursor movement. The single quotation mark (‘) syntax is used to move to the beginning of the line representing the previous context. The back quotation mark (`) syntax is used to move to the previous context *within* a line.

The Screen Commands

The screen commands are *not* cursor movement commands and cannot be used in delete commands as the delimiters of text objects. However, the screen commands do move the cursor and are useful in paging or scrolling through a file. These commands are described below:

Page – CTRL-U and CTRL-D

Syntax: [size]CTRL-U
 [size]CTRL-D

Function: Scrolls the screen up a half window (CTRL-U) or down a half window (CTRL-D). If *size* is given, then the scroll is *size* number of lines. This value is remembered for all later scrolling commands.

Scroll – CTRL-F and CTRL-B

Syntax: CTRL-F
 CTRL-B

Function: Pages screen forward and backward. Two lines of continuity are kept between pages if possible. A preceding *count* gives the number of pages to move forward or backward.

Status – CTRL-G

Syntax: **BELL**
CTRL-G

Function: Prints *vi* status on status line. This gives you the name of the file you are editing, whether it has been modified, the current line number, the number of lines in the file, and the percentage of the file (in lines) that precedes the cursor.

Zero Screen – z

Syntax: [*linenumber*]z[*size*]RETURN
[*linenumber*]z[*size*].
[*linenumber*]z[*size*]-

Function: Redraws the display with the current line placed at or “zeroed” at the top, middle, or bottom of the screen, respectively. If you give a *size*, then the number of lines displayed is equal to *size*. If a preceding *linenumber* is given, the given line is placed at the top of the screen. If the last argument is a RETURN, the current line is placed at the top of the screen. If the last argument is a period (.), the current line is placed in the middle of the screen. If the last argument is a minus sign (-), the current line is placed at the bottom of the screen.

Redraw – CTRL-R or CTRL-L

Syntax: **CTRL-R**
CTRL-L

Function: Redraws the screen. Use this command to erase any system messages that may scramble your screen. Note that system messages do not affect the file you are editing.

Text Insertion

The text insertion commands always place you in insert mode. Exit from insert mode is always done by pressing ESCAPE. The following insertion commands are “pure” insertion commands; no text is deleted when you use them. This differs from the text modification commands change, replace, and substitute, which delete and then insert text in one operation.

Insert – i and I

Syntax: **i**[*text*]ESCAPE
I[*text*]ESCAPE

Function: Insert *text* in editing buffer. The lowercase *i* command places you in insert mode. *Text* is inserted *before* the character beneath the cursor. To insert a NEWLINE, just press a RETURN. Exit insert mode by typing the ESCAPE key. The uppercase *I* command places you in insert mode, but begins text insertion at the beginning of the current line, rather than before the cursor.

Append – a and A

Syntax: a[*text*]ESCAPE
 A[*text*]ESCAPE

Function: Appends *text* to the editing buffer. The lowercase *a* command works exactly like the lowercase *i* command, except that text insertion begins after the cursor and not before. This is the one way to add text to the end of a line. The uppercase *A* command begins appending text at the end of the current line rather than after the cursor.

Open New Line – o and O

Syntax: o[*text*]ESCAPE
 O[*text*]ESCAPE

Function: Opens a new line and inserts text. The lowercase *o* command opens a new line below the current line; uppercase *O* opens a new line *above* the current line. After the new line has been opened, both these commands work like the *I* command.

Text Deletion

Many of the text deletion commands use the D key as an operator. This operator deletes text objects delimited by the cursor and a cursor movement command. Deleted text is always saved away in a buffer. The delete commands are described below:

Delete Character – x and X

Syntax: x
 X

Function: Deletes a character. The lowercase *x* command deletes the character beneath the cursor. With a preceding count, *count* characters are deleted to the right beginning with the character beneath the cursor. This is a quick and easy way to delete a few characters. The uppercase *X* command deletes the character just before the cursor. With a preceding *count*, *count* characters are deleted backward, beginning with the character just before the cursor.

Delete – d and D

Syntax: *d**cursor-movement*
dd
D

Function: Deletes a text object. The lowercase *d* command takes a *cursor-movement* as an argument. If the *cursor-movement* is an intraline command, then deletion takes place from the cursor to the end of the text object delimited by the *cursor-movement*. Deletion forward deletes the character beneath the cursor; deletion backward does not. If the *cursor-movement* is a multi-line command, then deletion takes place from and including the current line to the text object delimited by the *cursor-movement*.

The *dd* command deletes whole lines. The uppercase *D* command deletes from and including the cursor to the end of the current line.

Deleted text is automatically pushed on a stack of buffers numbered 1 through 9. The most recently deleted text is also placed in a special delete buffer that is logically buffer 0. This special buffer is the default buffer for all (put) commands using the double quotation mark (") to specify the number of the buffer for *delete*, *put*, and *yank* commands. The buffers 1 through 9 can be accessed with the *p* and *P* (put) commands by appending the double quotation mark (") to the number of the buffer. For example:

"4p

puts the contents of delete buffer number 4 in your editing buffer just below the current line. Note that the last deleted text is "put" by default and does not need a preceding buffer number.

Text Modification

The text modification commands all involve the replacement of text with other text. This means that some text will necessarily be deleted. All text modification commands can be "undone" with the *u* command:

Undo – u and U

Syntax: *u*
U

Function: Undoes the last insert or delete command. The lowercase *u* command undoes the last insert or delete command. This means that after an insert, *u* deletes text; and after a delete, *u* inserts text. For the purposes of undo, all text modification commands are considered insertions.

The uppercase *U* command restores the current line to its state before it was edited, no matter how many times the current line has been

edited since you moved to it.

Repeat – .

Syntax: .

Function: Repeats the last insert or delete command. A special case exists for repeating the *p* and *P* “put” commands. When these commands are preceded by the name of a delete buffer, then successive *u* commands print out the contents of the delete buffers.

Change – c and C

Syntax: *ccursor-movement text*ESCAPE
 *Ctext*ESCAPE
 *cctext*ESCAPE

Function: Changes a text object and replaces it with *text*. Text is inserted as with the *i* command. A dollar sign (\$) marks the extent of the change. The *c* command changes arbitrary text objects delimited by the cursor and a *cursor-movement*. The *C* and *cc* commands affect whole lines and are identical in function.

Replace – r and R

Syntax: *rchar*
 *Rtext*ESCAPE

Function: Overstrikes character or line with *char* or *text*, respectively. Use *r* to overstrike a single character and *R* to overstrike a whole line. A *count* multiplies the replacement text *count* times.

Substitute – s and S

Syntax: *stext*ESCAPE
 *Sstext*ESCAPE

Function: Substitutes current character or current line with *text*. Use *s* to replace a single character with new text. Use *S* to replace the current line with new text. If a preceding count is given, *text* substitutes for count number of characters or lines depending on whether the command is *s* or *S*, respectively.

Filter – !

Syntax: !*cursor-movement cmd* RETURN

Function: Filters the text object delimited by the cursor and *cursor-movement* through the XENIX command, *cmd*. For example, the following command sorts all lines between the cursor and the bottom of the screen, substituting the designated lines with the sorted lines:

!Lsort

Arguments and shell metacharacters may be included as part of *cmd*; however, standard input and output are always associated with the text object being filtered.

Join Lines – J

Syntax: **J**

Function: Joins the current line with the following line. If a *count* is given, then *count* lines are joined.

Shift – < and >

Syntax: >[*cursor-movement*]
 <[*cursor-movement*]
 >>
 <<

Function: Shifts text left (>) or right (<). Text is shifted by the value of the **shiftwidth** option, which is normally set to eight spaces. Both the > and < commands shift all lines in the text object delimited by the current line and *cursor-movement*. The >> and << commands affect whole lines. All versions of the command can take a preceding *count* that acts to multiply the number of objects affected.

Text Movement

The text movement commands move text in and out of the named buffers *a-z* and out of the delete buffers *1-9*. These commands either “yank” text out of the editing buffer and into a named buffer or “put” text into the editing buffer from a named buffer or a delete buffer. By default, text is put and yanked from the “unnamed buffer”, which is also where the most recently deleted text is placed. Thus, it is quite reasonable to delete text, move your cursor to the location where you want the deleted text placed, and put the text back into the editing buffer at this new location with the *p* or *P* command.

The named buffers are most useful for keeping track of several chunks of text that you want to keep on hand for later access, movement, or rearrangement. These buffers are named with the letters *a* through *z*. To refer to one of these buffers (or one of the numbered delete buffers) in a command, use a double quotation mark. For example, to yank a line into the buffer named *a*, type:

"ayy

To put this text back into the file, type:

"ap

If you delete text into the buffer named *A* rather than *a*, text is appended to the buffer.

Note that the contents of the named buffers are not destroyed when you switch files. Therefore, you can delete or yank text into a buffer, switch files, and do a put. Buffer contents are *destroyed* when you exit the editor, so be careful.

Put – p and P

Syntax: ["*alphanumeric*]p
 ["*alphanumeric*]P

Function: Puts text from a buffer into the editing buffer. If no buffer name is specified, then text is put from the unnamed buffer. The lowercase *p* command puts text either below the current line or after the cursor, depending on whether the buffer contains a partial line or not. The uppercase *P* command puts text either above the current line or before the cursor, again depending on whether the buffer contains a partial line or not.

Yank – y and Y

Syntax: ["*letter*]ycursor-movement
 ["*letter*]yy
 ["*letter*]Y

Function: Copies text in the editing buffer to a named buffer. If no buffer name is specified, then text is yanked into the unnamed buffer. If an upper-case *letter* is used, then text is appended to the buffer and does not overwrite and destroy the previous contents. When a *cursor-movement* is given as an argument, the delimited text object is yanked. The *Y* and *yy* commands yank a single line, or, if a preceding count is given, multiple lines can be yanked.

Searching

The search commands search either forward or backward in the editing buffer for text that matches a given regular expression.

Search – / and ?

Syntax: /[*pattern*]/[*offset*]RETURN
 /[*pattern*]RETURN
 ?[*pattern*?][*offset*]RETURN
 ?[*pattern*]RETURN

Function: Searches forward (/) or backward (?) for *pattern*. A string is actually a regular expression. The trailing delimiter is not required. If no *pattern* is given, then last *pattern* searched for is used. After the second delimiter, an *offset* may be given specifying the beginning of a line relative to the line on which *pattern* was found. For example:

/word/- finds the beginning of the line immediately preceding the line containing "word" and the following command:

/word/+2

finds the beginning of the line two lines after the line containing "word". See also the **ignorecase** and **magic** options.

Next String – n and N

Syntax: **n**
 N

Function: Repeats the last search command. The *n* command repeats the search in the same direction as the last search command. The *N* command repeats the search in the opposite direction of the last search command.

Find Character – f and F

Syntax: **fchar**
 Fchar
 ;
 ,

Function: Finds the *char* character on the current line. The lowercase *f* searches forward on the line; the uppercase *F* searches backward. The semicolon (;) repeats the last character search. The comma (,) reverses the direction of the search.

To Character – t and T

Syntax: **tchar**
 Tchar
 ;
 ,

Function: Moves the cursor up to but not on *char*. The semicolon (;) repeats the last character search. The comma (,) reverses the direction of the search.

Mark – m

Syntax: **m***letter*

Function: Marks a place in the file with a lowercase *letter*. You can move to a mark using the “to mark” commands described below. It is often useful to create a mark, move the cursor, and then delete from the cursor to the “a” mark with the following command:

d’a

To Mark – ‘ and `

Syntax: **’***letter*
`*letter*

Function: Move to *letter*. These commands let you move to the location of a mark. Marks are denoted by single lowercase alphabetic characters. Before you can move to a mark, it must first be created with the *m* command. The back quotation mark (**’**) moves you to the exact location of the mark within a line; the forward quotation mark (**`**) moves you to the beginning of the line containing the mark. Note that these commands are also legal cursor movement commands.

Exit and Escape Commands

There are several commands that are used to escape from *vi* command mode and to exit the editor. These are described in the following section.

Ex Escape – :

Syntax: **:**

Function: Enters *ex* escape mode to execute an *ex* command. The colon appears on the status line as a prompt for an *ex* command. You then can enter an *ex* command line terminated by either a RETURN or an ESCAPE and the *ex* command will execute. You are then prompted to type RETURN to return to *vi* command mode. During the input of the *ex* command line or during execution of the *ex* command, you may press INTERRUPT to stop what you are doing and return to *vi* command mode.

Exit Editor – ZZ

Syntax: **ZZ**

Function: Exit *vi* and write out the file if any changes have been made. This returns you to the shell from which you started *vi*.

Quit to Ex – Q

Syntax: Q

Function: Enters the *ex* editor. When you do this, you will still be editing the same file. You can return to *vi* by typing the *vi* command from *ex*.

Ex Commands

Typing the colon (:) escape command when in command mode produces a colon prompt on the status line. This prompt is for a command available in the line-oriented editor, *ex*. In general, *ex* commands let you write out or read in files, escape to the shell, or switch editing files. Many of these commands perform actions that affect the “current” file by default. The current file is normally the file that you named when you started *vi*, although the current file can be changed with the “file” command, *f*, or with the “next” command, *n*. In most respects, these commands are identical to similar commands for the editor, *ed*. All such *ex* commands are aborted by either RETURN or ESCAPE. We shall use RETURN in our examples. Command entry is terminated by typing INTERRUPT.

Command Structure

Most *ex* command names are English words, and initial prefixes of the words are acceptable abbreviations. In descriptions, only the abbreviation is discussed, since this is the most frequently used form of the command. The ambiguity of abbreviations is resolved in favor of the more commonly used commands. As an example, the *substitute* command can be abbreviated *s*, while the shortest available abbreviation for the *set* command is *se*.

Most commands accept prefix addresses specifying the lines in the file that they are to affect. A number of commands also may take a trailing *count* specifying the number of lines to be involved in the command. Counts are rounded down if necessary. Thus, the command “10p” will print the tenth line in the buffer, while “move 5” will move the current line after line 5.

Some commands take other information or parameters, stated after the command name. Examples might be option names in a *set* command, such as “set number”, a filename in an *edit* command, a regular expression in a *substitute* command, or a target address for a *copy* command. For example:

1,5 copy 25

A number of commands have variants. The variant form of the command is invoked by placing an exclamation mark (!) immediately after the command name. Some of the default variants may be controlled by options; in this case, the exclamation mark turns off the meaning of the default.

In addition, many commands take flags, including the “p” and “l” characters. A “p” or “l” must be preceded by a blank or TAB. In this case, the command abbreviated by these characters is executed after the command completes. Since *ex* normally prints the new current line after each change, *p* is rarely necessary. Any number of plus (+) or minus (–) characters may also be given with these flags. If they appear, the specified offset is applied to the current line value before the printing command is executed.

Most commands that change the contents of the editor buffer give feedback if the scope of the change exceeds a threshold given by the **report** option. This feedback helps to detect undesirably large changes so that they may be reversed quickly and easily with the *undo* command. After commands with global effect, you will be informed if the net change in the number of lines in the buffer during this command exceeds this threshold.

Command Addressing

The following list specifies the line addressing syntax for *ex* commands:

- The current line. Most commands leave the current line as the last line which they affect. The default address for most commands is the current line; thus, “.” is rarely used alone as an address.
- n* The *n*th line in the editor’s buffer, lines being numbered sequentially from 1.
- \$ The last line in the buffer.
- % An abbreviation for “1,\$”, the entire buffer.
- +*n* or –*n* An offset, *n* relative to the current buffer line. The forms “.+3”, “+3”, and “+++” are all equivalent. If the current line is line 100, they all address line 103.

/*pattern*/ or ?*pattern*?

Scan forward and backward respectively for a text matching the regular expression given by *pattern*. Scans normally wrap around the end of the buffer. If you only want to print the next line containing *pattern*, you may omit the trailing slash (/) or question mark (?). If *pattern* is omitted or explicitly empty, then the string matching the last specified regular expression is located. The forms “RETURN” and “?RETURN” scan a file using the last named regular expression. After a substitute, “RETURN” and “??RETURN” would scan the file using that substitute’s regular expression.

`` or `x

Before each nonrelative motion of the current line dot (.), the previous current line is marked with a label, subsequently referred to with two single quotation marks (``). This makes it easy to refer or return to this previous context. Marks are established with the *vi m* command, using a single lowercase letter as the name of the mark. Marked lines are later referred to with the notation:

`x.

where *x* is the name of a mark.

Addresses to commands consist of a series of addresses, separated by a colon (:) or a semicolon (;). Such address lists are evaluated left to right. When addresses are separated by a semicolon (;), the current line (.) is set to the value of the previous addressing expression before the next address is interpreted. If more addresses are given than the command requires, then all but the last one or two are ignored. If the command takes two addresses, the first addressed line must precede the second in the buffer. Null address specifications are permitted in a list of addresses, the default in this case is the current line “.”; thus, “.,100” is equivalent to “.,100”. It is an error to give a prefix address to a command which expects none.

Command Format

The following is the format for all *ex* commands:

[*address*] [*command*] [!] [*parameters*] [*count*] [*flags*]

All parts are optional depending on the particular command and its options. The following section describes specific commands.

Argument List Commands

The argument list commands allow you to work on a set of files, by remembering the list of filenames that are specified when you invoke *vi*. The *args* command lets you examine this list of filenames. The *file* command gives you information about the current file. The *n* (next) command lets you either edit the next file in the argument list or change the list. And the *rewind* command lets you restart editing the files in the list. All of these commands are described below:

args The members of the argument list are printed, with the current argument delimited by brackets. For example, a list might look like this:

file1 file2 [file3] file4 file5

The current file is *file3*.

- f** Prints the current filename, whether it has been modified since the last *write* command, whether it is read-only, the current linenumber, the number of lines in the buffer, and the percentage of the buffer that you have edited. In the rare case that the current file is "[Not edited]" this is noted also; in this case you have to use *w!* to write to the file, since the editor is not sure that a *w* command will not destroy a file unrelated to the current contents of the buffer.
- f file** The current filename is changed to *file* which is considered "[Not edited]".
- n** The next file in the command line argument list is edited.
- n!** This variant suppresses warnings about the modifications to the buffer not having been written out, discarding irretrievably any changes that may have been made.
- n [+command] filelist** The specified *filelist* is expanded, and the resulting list replaces the current argument list; the first file in the new list is then edited. If *command* is given (it must contain no spaces), it is executed after editing the first such file.
- rew** The argument list is rewound, and the first file in the list is edited.
- rew!** Rewinds the argument list, discarding any changes made to the current buffer.

Edit Commands

To edit a file other than the one you are currently editing, you will often use one of the variations of the *e* command.

In the following discussions, note that the name of the current file is always remembered by *vi* and is specified by a percent sign (%). The name of the *previous* file in the editing buffer is specified by a number sign (#).

The edit commands are described below:

- e file** Used to begin an editing session on a new file. The editor first checks to see if the buffer has been modified since the last *w* command was issued. If it has been, a warning is issued and the command is aborted. The command otherwise deletes the entire contents of the editor buffer, makes the named file the current file, and prints the new filename. After ensuring that this file is sensible, (i.e., that it is not a binary file, a directory, or a device), the editor reads the file into its buffer. If the read of the file completes without error, the number of lines and characters read is printed on the status line. If there were any non-ASCII characters in the file they are stripped of their non-ASCII high bits, and any null characters in the

file are discarded. If none of these errors occurred, the file is considered edited. If the last line of the input file is missing the trailing NEWLINE character, it is supplied and a complaint is issued. The current line is initially the first line of the file.

- e! file* This variant form suppresses the complaint about modifications having been made and not written from the editor buffer, thus discarding all changes that have been made before editing the new file.
- e +n file* Causes the editor to begin editing at line *n* rather than at the first line. The *n* argument may also be an editor command containing no spaces; for example, “+/pattern”.
- CTRL-^* This is a shorthand equivalent for “:e #RETURN”, which returns to the previous position in the last edited file. If you do not want to write the file you should use “:e! #RETURN” instead.

Write Commands

The write commands let you write out all or part of your editing buffer to either the current file or to some other file. These commands are described below:

- w file* Writes changes made back to *file*, printing the number of lines and characters written. Normally, *file* is omitted and the buffer is written to the name of the current file. If *file* is specified, text will be written to that file. The editor writes to a file only if it is the current file and is edited, or if the file does not exist. Otherwise, you must give the variant form *w!* to force the write. If the file does not exist it is created. The current filename is changed only if there is no current filename; the current line is never changed.

If an error occurs while writing the current and edited file, the editor prints:

No write since last change

even if the buffer had not previously been modified.

- w>> file* Appends the buffer contents at the end of an existing file. Previous file contents are not destroyed.
- w! name* Overrides the checking of the normal *write* command, and writes to any file that the system permits.
- w !command* Writes the specified lines into *command*. Note the difference between:

w! file

which overrides checks, and:

w !cmd

which writes to a command. The output of this command is displayed on the screen and not inserted in the editing buffer.

Read Commands

The read commands let you read text into your editing buffer at any location you specify. The text you read in must be at least one line long, and can be either a file or the output from a command.

r file Places a copy of the text of the given file in the editing buffer after the specified line. If no file is given then the current filename is used. The current filename is not changed unless there is none, in which case the file becomes the current name. If the file buffer is empty and there is no current name then this is treated as an *e* command.

Address 0 is legal for this command and causes the file to be read at the beginning of the buffer. Statistics are given as for the *e* command when the *r* successfully terminates. After an *r* the current line is the last line read.

r !command Reads the output of *command* into the buffer after the specified line. A space or tab before the exclamation mark (!) is mandatory.

Quit Commands

There are several ways to exit *vi*. Some abort the editing session, some write out the editing buffer before exiting, and some warn you if you decide to exit without writing out the buffer. All of these ways of exiting are described in the following list:

q Exits *vi*. No automatic write of the editor buffer to a file is performed. However, *vi* issues a warning message if the file has changed since the last *w* command was issued, and does not quit. *Vi* will also issue a diagnostic if there are more files in the argument list left to edit. Normally, you will wish to save your changes, and you should give a *w* command. If you wish to discard them, use the *q!* command variant.

q! Quits from the editor, discarding changes to the buffer without complaint.

wq name Like a *w* and then a *q* command.

wq! name Overrides checking normally made before execution of the *w* command to any file. For example, if you own a file but do not have write permission turned on, the *wq!* allows you to update the file anyway.

x name If any changes have been made and not written, writes the buffer out and then quits. Otherwise, it just quits.

Global and Substitute Commands

The global and substitute commands allow you to perform complex changes to a file in a single command. Learning how to use these commands is a must for the serious user of *vi*.

g/pattern/cmds

The *g* command has two distinct phases. In the first phase, each line matching *pattern* in the editing buffer is marked. Next, the given command list is executed with the current line, dot (*.*), initially set to each marked line.

The command list consists of the remaining commands on the current input line and may continue to multiple lines by ending all but the last such line with a backslash (**). This multiple-line option will not work from within *vi*, you must switch to *ex* to do it. If *cmds* (or the trailing slash (*/*) delimiter) is omitted, then each line matching *pattern* is printed.

The *g* command itself may not appear in *cmds*. The **autoprint** and **autoindent** options are inhibited during a global command and the value of the **report** option is temporarily infinite, in deference to a **report** for the entire global. Finally, the context mark (*`*) or (*^*) is set to the value of the current line (*.*) before the global command begins and is not changed during a global command.

The following global commands, most of them substitutions, cover the most frequent uses of the global command.

- | | |
|----------------------|---|
| g/s1/p | This command simply prints all lines that contain the string "s1". |
| g/s1/s1/s2/ | This command substitutes the <i>first</i> occurrence of "s1" on all lines that contain it with the string "s2". |
| g/s1/s1/s2/g | This command substitutes all occurrences of "s1" with the string "s2". This includes multiple occurrences of "s1" on a line. |
| g/s1/s1/s2/gp | This command works the same as the preceding example, except that all changed lines are printed on the screen. |
| g/s1/s1/s2/gc | This command asks you to confirm that you want to make each substitution of the string "s1" with the string "s2". If you type a "Y" then the given substitution is made, otherwise it is not. |

g/s0/s1/s2/g This command marks all those lines that contain the string "s0", and for those lines only, it substitutes all occurrences of the string "s1" with "s2".

g!/pattern/cmds This variant form of *g* runs *cmds* at each line that doesn't match *pattern*.

s/pattern/repl/options

On each specified line, the first instance of text matching the regular expression *pattern* is replaced by the replacement text *repl*. If the *global* indicator option character *g* appears, then all instances on a line are substituted. If the *confirm* indication character, *c*, appears, then before each substitution the line to be substituted is printed on the screen with the string to be substituted marked with caret (^) characters. By pressing the Y key, you perform the substitution; any other input causes no change to take place. After an *s* command the current line is the last line substituted.

v/pattern/cmds A synonym for the *global* command variant *g!*, running the specified *cmds* on each line that does not match *pattern*.

Text Movement Commands

The text movement commands are largely superseded by commands available in *vi* command mode. However, the following two commands are still quite useful:

co *addr flags* A copy of the specified lines is placed after *addr*, which may be "0". The current line "." addresses the last line of the copy.

[*range*]*maddr* The *m* command moves the lines specified by *range* after the line given by *addr*. For example, *m+* swaps the current line and the following line, since the default range is just the current line. The first of the moved lines becomes the current line (⌘ot).

Shell Escape Commands

You will often want to escape from the editor to execute normal XENIX commands. You may also want to change your working directory so that your editing can be done with respect to a different working directory. These operations are described below:

cd *directory* The specified *directory* becomes the current directory. If no directory is specified, the current value of the *home* option is used as the target directory. After a *cd*, the current file is not considered to have been edited so that write restrictions on preexisting files still apply.

sh A new shell is created. You may invoke as many commands as you like in this shell. To return to *vi*, type a CTRL-D to terminate the shell.

!command The remainder of the line after the exclamation (!) is sent to a shell to be executed. Within the text of *command* the characters “%” and “#” are expanded as the filenames of the current file and the last edited file and the character “!” is replaced with the text of the previous command. Thus, in particular, “!!” repeats the last such shell escape. If any such expansion is performed, the expanded line is echoed. The current line is unchanged by this command.

If there has been “[No write]” of the buffer contents since the last change to the editing buffer, a diagnostic is printed before the command is executed as a warning. A single exclamation (!) is printed when the command completes.

Other Commands

The following command descriptions explain how to use miscellaneous *ex* commands that do not fit into the above categories:

abbr Maps the first argument to the following string. For example, the following command

```
:abbr rainbow yellow green blue red
```

maps “rainbow” to “yellow green blue red”. Abbreviations can be turned off with the *unabbreviate* command, as in:

```
:una rainbow
```

map, map! Maps any character or escape sequence to an existing command sequence. Characters mapped with *map!* work in both command and insert mode, while characters mapped with *map* work only in command mode. Characters mapped with *map!* cannot be unmapped using *unmap*.

nu Prints each specified line preceded by its buffer line number. The current line is left at the last line printed. To get automatic line numbering of lines in the buffer, set the **number** option.

preserve The current editor buffer is saved as though the system had just crashed. This command is for use only in emergencies when a *w* command has resulted in an error and you don’t know how to save your work.

= Prints the total number of lines in the file. The current line is unchanged.

recover file

Recovers *file* from the system save area. The system saves a copy of the editing buffer only if you have made changes to the file, the system crashes, or you execute a *preserve* command. When you use *preserve* you will be notified by mail when a file is saved.

set argument

With no arguments, *set* prints those options whose values have been changed from their defaults; with the argument *all*, it prints all of the option values.

Giving an option name followed by a question mark (?) causes the current value of that option to be printed. The question mark is unnecessary unless the option is a Boolean value. Switch options are given values either with:

set option

to turn them on, or:

set nooption

to turn them off. String and numeric options are assigned with:

set option=value

More than one parameter may be given to *set*; all are interpreted from left to right.

tag label

The focus of editing switches to the location of *label*. If necessary, *vi* will switch to a different file in the current directory to find *label*. If you have modified the current file before giving a *tag* command, you must first write it out. If you give another *tag* command with no argument, the previous *label* is used.

Similarly, if you type only a CTRL-], *vi* searches for the word immediately after the cursor as a tag. This is equivalent to typing “:tag”, the word following the cursor, and then pressing the RETURN key.

The tags file is normally created by a program such as *ctags*, and consists of a number of lines with three fields separated by spaces or tabs. The first field gives the name of the tag, the second the name of the file where the tag resides, and the third gives an addressing form which can be used by the editor to find the tag. This field is usually a contextual scan using */pattern/* to be immune to minor changes in the file. Such scans are always performed as if the *nomagic* option was set. The tag names in the tags file must be sorted alphabetically. There are a number of options that can be set to affect the *vi* environment. These can be set with the *ex set* command either while editing or immediately after *vi* is invoked in the *vi* start-up file, *.exrc*.

unmap

Unmaps any character or escape sequence that has been mapped using the *map* command.

The first thing that must be done before you can use *vi*, is to set the terminal type so that *vi* understands how to talk to the particular terminal you are using.

Each time *vi* is invoked, it reads commands from the *.exrc* file in your home directory. This file normally sets the user's preferred options so that they need not be set manually each time you invoke *vi*. Each of the options is described in detail below.

Options

There are only two kinds of options: switch options and string options. A switch option is either on or off. A switch is turned off by prefixing the word *no* to the name of the switch within a *set* command. String options are strings of characters that are assigned values with the syntax: *option=string*. Multiple options may be specified on a line. *Vi* options are listed below:

autoindent, ai default: **noai**

Can be used to ease the preparation of structured program text. For each line created by an append, change, insert, open, or substitute operation, *vi* looks at the preceding line to determine and insert an appropriate amount of indentation. To back the cursor up to the preceding tab stop, you can type CTRL-D. The tab stops going backward are defined as multiples of the *shiftwidth* option. You cannot BACKSPACE over the indent, except by typing a CTRL-D.

Specially processed in this mode is a line with no characters added to it, which turns into a completely blank line (the white space provided for the *autoindent* is discarded.) Also specially processed in this mode are lines beginning with a caret (^) and immediately followed by a CTRL-D. This causes the input to be repositioned at the beginning of the line, retaining the previous indent for the next line. Similarly, a "0" followed by a CTRL-D repositions the cursor at the beginning without retaining the previous indent. *Autoindent* doesn't happen in global commands.

autoprint ap default: **ap**

Causes the current line to be printed after each *ex copy*, *move*, or *substitute* command. This has the same effect as supplying a trailing "p" to each such command. *Autoprint* is suppressed in globals, and only applies to the last command on a line.

autowrite, aw default: **noaw**

Causes the contents of the buffer to be automatically written to the current file if you have modified it when you give a *next*, *rewind*, *tag*, or *!* command, or a CTRL-^ (switch files) or CTRL-] (tag goto) command.

beautify, bf default: **nobeautify**

Causes all control characters except TAB, NEWLINE and FORMFEED to be discarded from the input. A complaint is registered the first time a BACKSPACE character is discarded. **Beautify** does not apply to command input.

directory, dir default: **dir=/tmp**

Specifies the directory in which *vi* places the editing buffer file. If this directory is not writable, then the editor will exit abruptly when it fails to write to the buffer file.

edcompatible default: **noedcompatible**

Causes the presence or absence of *g* and *c* suffixes on substitute commands to be remembered, and to be toggled on and off by repeating the suffixes. The *r* suffix causes the substitution to be like the tilde (~) command, instead of like the ampersand command (&).

errorbells, eb default: **noeb**

Error messages are preceded by a bell. If possible, the editor always places the error message in reverse video instead of ringing the bell.

hardtabs, ht default: **ht=8**

Gives the boundaries on which terminal hardware tabs are set or on which the system expands TABs.

ignorecase, ic default: **noic**

Maps all uppercase characters in the text to lowercase in regular expression matching. In addition, all uppercase characters in regular expressions are mapped to lowercase except in character class specifications enclosed in brackets.

lisp default: **noisp**

Autoindent indents appropriately for LISP code, and the () { } [] and] commands are modified to have meaning for LISP.

list default: **noist**

All printed lines will be displayed unambiguously, showing tabs and end-of-lines.

magic default: **magic**

If **nomagic** is set, the number of regular expression metacharacters is greatly reduced, with only caret (^) and dollar sign (\$) having special effects. In addition, the metacharacters "-" and "&" in replacement patterns are treated as normal characters. All the normal metacharacters may be made **magic** when **nomagic** is set by preceding them with a backslash (\). See the following section, "Regular Expressions", for a more complete explanation of the regular expression metacharacters and the **magic** option.

mesg default: **nomesg**

Causes write permission to the terminal to be turned off while you are in visual mode, if **nomesg** is set. This prevents people from writing to your screen with the XENIX *write* command.

number, n default: **nonumber**

Causes all output lines to be printed with their line numbers.

open default: **open**

If set to **noopen**, the commands *open* and *visual* are not permitted from *ex*. This is set to prevent confusion resulting from accidental entry to open or visual mode.

optimize, opt default: **optimize**

Output of text to the screen is expedited by setting the terminal so that it does not perform automatic RETURNS when printing more than one line of output, thus greatly speeding output on terminals without addressable cursors when text with leading white space is printed.

paragraphs, para default: **para=IPLPPPQPP TPbp**

Specifies paragraph delimiters for the { and } operations. The pairs of characters in the option's value are the names of the *nroff* macros that start paragraphs.

prompt default: **prompt**

Ex input is prompted for with a colon (:). If **noprompt** is set when *ex* command mode is entered with the *Q* command, no colon prompt is displayed on the status line.

redraw default: **noredraw**

The editor simulates (using great amounts of output) an intelligent terminal on a "dumb" terminal. Useful only at very high speed.

remap default: **remap**

If on, mapped characters are repeatedly tried until they are unchanged. For example, if *o* is mapped to *O* and *O* is mapped to *I*, *o* will map to *I* if **remap** is set, and to *O* if **noremap** is set.

report default: **report=5**

Specifies a threshold for feedback from commands. Any command that modifies more than the specified number of lines will provide feedback about the scope of its changes. For *global* commands and the *undo* command, which have potentially far reaching scope, the net change in the number of lines in the buffer is presented at the end of the command, subject to this same threshold. Thus, notification is suppressed during a *g* command on the individual commands performed.

scroll default: **scroll=½ window**

Determines the number of logical lines scrolled when CTRL-D is received from a terminal input in command mode, and the number of lines printed by a command mode *z* command (double the value of *scroll*).

sections default: **sections=SHNHH HU**

Specifies the section macros for the [[and]] operations. The pairs of characters in the option's value are the names of the *nroff* macros that start paragraphs.

shell, sh default: **sh=/bin/sh**

Gives the pathname of the shell forked for the shell escape command (!), and by the *shell* command. The default is taken from SHELL in the environment, if present.

shiftwidth, sw default: **sw=8**

Gives the width of a software tab stop, used in reverse tabbing with CTRL-D when using *autoindent* to append text, and by the *shift* commands.

showmatch, sm default: **nosm**

When a) or } is typed, moves the cursor to the matching (or { for one second if this matching character is on the screen.

tabstop, ts default: **ts=8**

The editor expands TABs in the input file to be on *n* boundaries for the purposes of display.

taglength, tl default: **tl=0**

The first *n* characters in a tag name are significant, but all others are ignored. A value of zero (the default) means that all characters are significant.

tags default: **tags=/usr/lib/tags**

A path of files to be used as tag files for the *tag* command. A requested tag is searched for in the specified files, sequentially. By default, files named **tag** are searched for in the current directory and in **/usr/lib**.

term default=**value of shell TERM variable**

The terminal type of the output device.

terse default: **noterse**

Shorter error diagnostics are produced for the experienced user.

warn default: **warn**

Warn if there has been "[No write since last change]" before a shell escape command (!).

window default: **window = speed dependent**

This specifies the number of lines in a text window. The default is 8 at slow speeds (600 baud or less), 16 at medium speed (1200 baud), and the full screen (minus one line) at higher speeds.

w300, w1200, w9600

These are not true options but set **window** only if the speed is slow (300), medium (1200), or high (9600), respectively.

wrapscan, ws default: **ws**

Searches using the regular expressions in addressing will wrap around past the end of the file.

wrapmargin, wm default: **wm=0**

Defines the margin for automatic insertion of newlines during text input. A value of zero specifies no wrap margin.

writeany, wa default: **nowa**

Inhibits the checks normally made before *write* commands, allowing a write to any file that the system protection mechanism will allow.

Regular Expressions

A regular expression specifies a set of strings of characters. A member of this set of strings is said to be “matched” by the regular expression. *Vi* remembers two previous regular expressions: the previous regular expression used in a substitute command and the previous regular expression used elsewhere, referred to as the previous *scanning* regular expression. The previous regular expression can always be referred to by a null regular expression: e.g., “/” or “??”.

The regular expressions allowed by *vi* are constructed in one of two ways depending on the setting of the **magic** option. The *ex* and *vi* default setting of **magic** gives quick access to a powerful set of regular expression metacharacters. The disadvantage of **magic** is that the user must remember that these metacharacters are **magic** and precede them with the backslash (\) to use them as “ordinary” characters. With **nomagic** set, regular expressions are much simpler, there being only two metacharacters. The power of the other metacharacters is still available by preceding the now ordinary character with a “\”. Note that “\” is always a metacharacter. In this discussion the **magic** option is assumed. With **nomagic** the only special characters are the caret (^) at the beginning of a regular expression, the dollar sign (\$) at the end of a regular expression, and the backslash (\). The tilde (~) and the ampersand (&) also lose their special meanings related to the replacement pattern of a substitute.

The following basic constructs are used to construct **magic** mode regular expressions:

char An ordinary character matches itself. Ordinary characters are any characters except a caret (^) at the beginning of a line, a dollar sign (\$) at the end of a line, a star (*) as any character other than the first, and any of the following characters:

. \ []

These characters must be preceded by a backslash (\) if they are to be treated as ordinary characters.

At the beginning of a pattern this forces the match to succeed only at the beginning of a line.

- \$ At the end of a regular expression this forces the match to succeed only at the end of the line.
- .
- \< Forces the match to occur only at the beginning of a “word”; that is, either at the beginning of a line, or just before a letter, digit, or underline and after a character not one of these.
- \> Similar to “\<”, but matching the end of a “word”, i.e. either the end of the line or before a character which is not a letter, a digit, or an underline character.

[*string*]

Matches any single character in the class defined by *string*. Most characters in *string* define themselves. A pair of characters separated by a dash (–) in *string* defines the set of characters between the specified lower and upper bounds, thus “[a–z]” as a regular expression matches any single lowercase letter. If the first character of *string* is a caret (^), the construct matches those characters which it otherwise would not. Thus, “[^a–z]” matches anything but a lowercase letter or a NEWLINE. To place any of the caret, left bracket, or dash characters in *string* they must be escaped with a preceding backslash (\).

The concatenation of two regular expressions first matches the leftmost regular expression and then the longest string that can be recognized as a regular expression. The first part of this new regular expression matches the first regular expression and the second part matches the second. Any of the single character matching regular expressions mentioned above may be followed by a star (*) to form a regular expression that matches zero or more adjacent occurrences of the characters matched by the prefixing regular expression. The tilde (~) may be used in a regular expression to match the text that defined the replacement part of the last *s* command. A regular expression may be enclosed between the sequences “\(`” and “\)`” to remember the text matched by the enclosed regular expression. This text can later be interpolated into the replacement text using the following notation:

\digit

where *digit* enumerates the set of remembered regular expressions.

The basic metacharacters for the replacement pattern are the ampersand (&) and the tilde (~); these are given as “\&” and “\~” when *nomagic* is set. Each occurrence of the ampersand is replaced by the characters matched by the regular expression. In the replacement pattern the tilde stands for the text of the previous replacement pattern.

Other possible metasequences in the replacement pattern are always introduced by a backslash (\). The sequence “\n” is replaced by the text matched by the *n*th regular subexpression enclosed between “\(`” and “\)`”. When nested, parenthesized subexpressions are present, *n* is determined by counting occurrences of “\(`” starting from the left. The sequences “\u” and “\l” cause the immediately following character in the replacement to be converted to uppercase or lowercase,

respectively, if this character is a letter. The sequences “\U” and “\L” turn such conversion on, until either “\E” or “\e” is encountered, or until the end of the replacement pattern.

Limitations

When using *vi* you should note the following limits:

250K bytes in a file

512 characters per line

255 characters per global command list

128 characters per filename

128 characters in the previous inserted and deleted text

128 characters in a shell escape command

63 characters in a string valued option

32 characters in a tag name

Approximately 14000 empty lines per file where an empty line is 1 character long

Notes

The */usr/lib/ex3.7/preserve* program can be used to restore *vi* buffer files that were lost as a result of a system crash. The program searches the */tmp* directory for *vi* buffer files and places them in the */usr/preserve* directory. The owner can retrieve these files using the *-r* option.

The */usr/lib/ex3.7/preserve* program must be placed in the system startup file, */etc/rc*, before the command that cleans out the */tmp* directory. See the *XENIX Operations Guide* for more information on */etc/rc*.

This utility was developed at the University of California at Berkeley and is used with permission.

Bugs

The “filter” command (!) may produce unpredictable results when operating on very large pieces of text.

Name

`vsh` – Invokes the visual shell.



Syntax

`vsh options`

Description

Vsh is a visual command interpreter with an interface much like that of other Microsoft tools. This shell is recommended for casual users of the system and those with specialized needs that do not include learning the complete operating system. Note that the visual shell can be substituted for a user's normal login shell by altering the `/etc/passwd` file.

See Also

`csh`(CP), `sh`(C)

XENIX Visual Shell User's Guide



Bugs

When running `vsh` on a remote terminal, the cursor disappears.



Name

`wait` – Awaits completion of background processes.

Syntax

`wait`

Description

Waits until all background processes started with an ampersand (&) have finished and reports on abnormal terminations.

Because the *wait(S)* system call must be executed in the parent process, the shell itself executes *wait*, without creating a new process.

See Also

`sh(C)`

Notes

Not all of the processes of a pipeline with three or more stages are children of the shell, and cannot be waited for.



Name

wall – Writes to all users.

Syntax

/etc/wall

Description

Wall reads a message from the standard input until an end-of-file is given. It then sends this message to all users currently logged in preceded by “Broadcast Message from ...”. *Wall* is used to warn all users, for example, prior to shutting down the system.

The sender should be super-user to override any protections the users may have invoked.

Files

/dev/tty*

See Also

mesg(C), write(C)

Diagnostics

Cannot send to ... The open on a user’s tty file has failed.



Name

`wc` – Counts lines, words, and characters.

Syntax

`wc [ -lwc ] [ names ]`

Description

`Wc` counts lines, words and characters in the named files, or in the standard input if no *names* appear. It also keeps a total count for all named files. A word is a maximal string of characters delimited by SPACEBARs, TABs, or NEWLINEs.

The `-l`, `-w`, and `-c` options may be used in any combination to specify that a subset of lines, words, and characters are to be reported. The default is `-lwc`.

When *names* are specified on the command line, they will be printed along with the counts.



Name

what – Identifies files.

Syntax

what *files*

Description

What searches the given files for all occurrences of the `@(#)` pattern and prints out what follows until the first tilde (`~`), greater-than sign (`>`), NEWLINE, backslash (`\`), or null character. The SCCS command, *get*(CP), substitutes this string as part of the `%Z%` string.

For example, if the shell procedure in file **print** contains:

```
# @(#)this is the print program
# @(#)syntax: print [files]
pr $* | lpr
```

then the command:

what **print**

displays the name of the file, **print**, and the identifying strings in that file:

```
print:
      this is the print program
      syntax: print [files]
```

What is intended to be used with the *get*(CP) command, which automatically inserts identifying information, but it can also be used where the information is inserted manually.

See Also

admin(CP), *get*(CP)



Name

`who` – Lists who is on the system.

Syntax

`who [ -adHlpqsTtu ] [ file ]`

`who am i`

`who am I`

Description

Who can list the user's name, terminal line, login time, elapsed time since activity occurred on the line, and the process ID of the command interpreter (shell) for each current XENIX system user. It examines the `/etc/utmp` file to obtain its information. If *file* is given, that file is examined. Usually, *file* will be `/etc/wtmp`, which contains a history of all the logins since the file was last created.

Who with the `am i` or `am I` option identifies the invoking user.

Except for the default `-s` option, the general format for output entries is:

name [*state*] *line* *time* *activity* *pid* [*comment*] [*exit*]

With options, *who* can list logins, logoffs, reboots, and changes to the system clock, as well as other processes spawned by the *init* process. These options are:

- `-a` This option processes the `/etc/utmp` file or the named *file* with options `-s`, `-t`, `-p`, `-l`, `-u`, and `-d` turned on.
- `-d` This option displays all processes that have expired and not been respawned by *init*. The *exit* field appears for dead processes and contains the termination and exit values (as returned by *wait(S)*), of the dead process. This can be useful in determining why a process terminated.
- `-H` This option will print column headings above the regular output.
- `-l` This option lists only those lines on which the system is waiting for someone to login. The *name* field is `LOGIN` in such cases. Other fields are the same as for user entries except that the *state* field does not exist.
- `-p` This option displays all processes except *getty* and users.
- `-q` This is a quick *who*, displaying only the names and the number of users currently logged on. When this option is used, all other options are ignored.

- s This option is the default and lists only the NAME, LINE and TIME fields.
- T This option is the same as the -u option, except that the *state* of the terminal line is printed. The *state* describes whether someone else can write to that terminal. A plus character (+) appears if the terminal is writable by anyone; a minus character (-) appears if it is not. The super-user can write to all lines having a plus character (+) or a minus character (-) in the *state* field. If a bad line is encountered, a question mark (?) is printed.
- t This option indicates the last change to the system clock (via the *date(C)* command) by the super-user. See *su(C)*.
- u This option lists only those users who are currently logged in. The *name* is the user's login name. The *line* is the name of the line as found in the */dev* directory. The *time* is the time that the user logged in. The *activity* is the number of hours and minutes since activity last occurred on that particular line. A dot (.) indicates that the terminal has seen activity in the last minute and is therefore "current." If more than twenty-four hours have elapsed or the line has not been used since boot time, the entry is marked old. This field is useful when trying to determine whether a person is working at the terminal or not. The *pid* is the process ID of the user's shell. The *comment* is the comment field. It can contain information about where the terminal is located, the telephone number of the dataset, the type of terminal if hard-wired, etc.

Files

/etc/utmp
/etc/wtmp

See Also

date(C), *login(C)*, *mesg(C)*, *su(C)*, *utmp(F)*, *wait(S)*

Name

whodo – Determines who is doing what.

Syntax

/etc/whodo

Description

Whodo produces merged, reformatted, and dated output from the *who*(C) and *ps*(C) commands.

See Also

ps(C), who(C)



Name

`write` – Writes to another user.

Syntax

`write user [ tty ]`

Description

Write copies lines from your terminal to that of another user. When first called, it sends the message:

Message from *your-logname* *your-tty* ...

The recipient of the message should write back at this point. Communication continues until an end-of-file character is read from the terminal or an interrupt is sent. At that point, *write* displays:

(end of message)

on the other terminal and exits.

If you want to write to a user who is logged in more than once, the *tty* argument may be used to indicate the appropriate terminal.

Permission to write may be denied or granted by use of the *mesg*(C) command. At the outset, writing is allowed. Certain commands, in particular *nroff*(CT) and *pr*(C), disallow messages in order to prevent messy output.

If the `!` character is found at the beginning of a line, *write* calls the shell to execute the rest of the line as a command.

The following protocol is suggested for using *write*: when you first write to another user, wait for him or her to write back before starting to send. Each party should end each message with a distinctive signal ((`o`) for “over” is conventional), indicating that the other may reply; (`oo`) for “over and out” is suggested when conversation is to be terminated.

Files

<code>/etc/utmp</code>	To find user
<code>/bin/sh</code>	To execute <code>!</code>

WRITE(C)

WRITE(C)

See Also

mail(C), mesg(C), who(C)

Name

xargs – Constructs and executes commands.

Syntax

```
xargs [flags] [command [initial-arguments ] ]
```

Description

Xargs combines the fixed *initial-arguments* with arguments read from the standard input to execute the specified *command* one or more times. The number of arguments read for each *command* invocation and the manner in which they are combined are determined by the flags specified.

Command, which may be a shell file, is searched for using the shell `$PATH` variable. If *command* is omitted, `/bin/echo` is used.

Arguments read in from standard input are defined as contiguous strings of characters delimited by one or more SPACEBARs TABs, or NEWLINES; empty lines are always discarded. SPACEBARs and TABs may be embedded as part of an argument if escaped or quoted: Characters enclosed in quotation marks (single or double) are taken literally, and the delimiting quotation marks are removed. Outside of quoted strings, a backslash (\) will escape the next character.

Each argument list is constructed starting with the *initial-arguments*, followed by some number of arguments read from standard input (exception: see `-i` flag). The `-i`, `-l`, and `-n` flags determine how arguments are selected for each command invocation. When none of these flags are coded, the *initial-arguments* are followed by arguments read continuously from standard input until an internal buffer is full, and then *command* is executed with the accumulated *args*. This process is repeated until there are no more *args*. When there are flag conflicts (e.g., `-l` vs. `-n`), the last flag has precedence. *Flag* values are:

- `-nnumber`** *Command* is executed for each *number* lines of nonempty arguments from the standard input. This is instead of the default single line of input for each *command*. The last invocation of *command* will be with fewer lines of arguments if fewer than *number* remain. A line is considered to end with the first NEWLINE *unless* the last character of the line is a SPACEBAR or a TAB; a trailing SPACEBAR/TAB signals continuation through the next nonempty line. If *number* is omitted, 1 is assumed. The `-x` option is forced.
- `-ireplstr`** Insert mode: *command* is executed for each line from the standard input, taking the entire line as a single *arg*, and inserting it in *initial-arguments* for each occurrence of *replstr*. A maximum of five arguments in *initial-arguments* may each contain one or more instances of *replstr*. SPACEBARs and TABs at the beginning of each line are thrown away. Constructed arguments may not grow larger than 255 characters, and the `-x` option is also forced. { } is assumed

for *replstr* if not specified.

- nnumber** Executes *command* using as many standard input arguments as possible, up to *number* arguments maximum. Fewer arguments will be used if their total size is greater than *size* characters, and for the last invocation if there are fewer than *number* arguments remaining. If the **-x** option is also coded, each *number* arguments must fit in the *size* limitation, or else *xargs* terminates execution.
- t** Trace mode: The *command* and each constructed argument list are echoed to file descriptor 2 just prior to their execution.
- p** Prompt mode: The user is asked whether to execute *command* each invocation. Trace mode (**-t**) is turned on to print the command to be executed, followed by a *?...* prompt. A reply of "y" (optionally followed by anything) will execute the command; anything else, including just a RETURN, skips that particular invocation of *command*.
- x** Causes *xargs* to terminate if any argument list would be greater than *size* characters; **-x** is forced by the **-i** and **-l** options. When none of the **-i**, **-l**, or **-n** options are coded, the total length of all arguments must be within the *size* limit.
- ssize** The maximum total size of each argument list is set to *size* characters; *size* must be a positive integer less than or equal to 470. If **-s** is not coded, 470 is taken as the default. Note that the character count for *size* includes one extra character for each argument and the count of characters in the command name.
- eeofstr** *Eofstr* is taken as the logical end-of-file string. Underscore (*_*) is assumed for the logical EOF string if **-e** is not coded. **-e** with no *eofstr* coded turns off the logical EOF string capability (underscore is taken literally). *Xargs* reads standard input until either end-of-file or the logical EOF string is encountered.

Xargs terminates if it either receives a return code of **-1** from *command*, or if it cannot execute *command*. When *command* is a shell program, it should explicitly *exit* (see *sh*(C)) with an appropriate value to avoid returning accidentally with **-1**.

Examples

The following example will move all files from directory \$1 to directory \$2, and echo each move command just before doing it:

```
ls $1 | xargs -i -t mv $1/{ } $2/{ }
```

Name

xmail – An Improved Mail Program.

Syntax

xmail [*options*] [*user...*]

Description

Xmail is an improved version of the mail program. It lets you use any editor available on your XENIX system for creating your *xmail* messages. *Xmail* also maintains a local working mailbox in which past messages and new mail can be kept for later referencing.

If you simply wish to read your mail, type:

xmail [*options*]

The available options are:

- d** Overrides the initialization file *.xmailrc* “*maildir=*” option.
- q** Sets “quiet” mode when you are not sending mail. In this mode, *xmail* does not display the messages in your file when it starts up or when you enter a new mailbox. This is the same as “*set -showmsgs*” in *.xmailrc*.

When used while sending mail, the *-q* option sets “quick” mode. When “quick” mode is set, *xmail* does not automatically enter the editor, regardless of how the “*editmail*” option is set. Instead, *xmail* prompts for “Subject” and other information without invoking the editor.
- R** Makes the mailfile “readonly”. That is, you cannot get new mail, delete messages, or edit messages. When *-R* is used, *xmail* does not make a working copy of the mailbox and therefore loads the mailbox faster. This option is useful if you only wish to look at the mailbox.
- f *mailfile*** Opens *mailfile* as the mailbox instead of the default mailbox.
- m** Opens a *mailbox* file in the user’s mail directory instead of the current working directory (used with *-f* option).
- s *subject*** Sets the subject line to *subject* when sending mail.

xmail.rc File

When you invoke *xmail*, it reads the file `/usr/lib/xmail.rc`. *Xmail* gets the initialization options for the system from this file. Next, *xmail* tries to open the `.xmailrc` file in your home directory. From this file, *xmail* gets options that you set. You can set or change these options temporarily using the *xmail* “o” command or permanently by editing the `.xmailrc` file.

The options are:

`alias mailname username [username...]`

Creates a user alias. This sets *mailname* to one or more user addresses.

`fullname=name`

`fullname=OFF`

Allows you to specify your full name for *xmail* to use when sending network mail. When *xmail* sends network mail, it adds this line to the message:

From: *system!user* (Full Name Here)

Use the `fullname` option to change the name shown or specify `OFF` if you don't want your full name used.

`ignore userid [userid...]`

Instructs *xmail* to automatically delete any mail coming from a user whose login name is *userid*. To display the current ignore list, type at the `mail>` prompt:

o show ignores

`mailbox=name`

Changes the name of the default working mailbox from `.mbox` to *name*. *Xmail* opens *name* in your home directory.

`maildir=directory`

Changes the current working directory when you run *xmail* for the duration of the mail session. Note that “`xmail -d`” overrides this option.

`record_file=filename`

`record_file=OFF`

Tells *xmail* to save a copy of all mail you send out in *filename*. This option acts independent of `autocc`. This option differs from the `autocc` option in that the message is not sent through the system mailer; but is, instead, copied into the specified file.

`savembox=name`

Changes the name of the default save mailbox from `mbox` to *name*.

set [-]option

Sets specific parameters for *xmail*. Possible options are given next.

Note: Specify the minus sign (-) to turn the option off.

autocc	Tells <i>xmail</i> to automatically send you a carbon copy (Cc:) of all mail you send.
autodelete	Tells <i>xmail</i> to automatically delete all messages in your current mailbox when you quit <i>xmail</i> . <i>Xmail</i> only deletes messages if you are in your default working mailbox. Since this is a potentially dangerous command, <i>xmail</i> asks if you really wish to quit when you type "q" to quit. If you select the Q command (Abort, no update), <i>xmail</i> does not delete any messages.
autoprint	Displays the next message automatically when you delete or save the current message.
banner	Displays a banner saying " <i>user's</i> Mail" when printing mail.
clear_hdr	Clears the screen when displaying message headers using the header command (h), or when displaying option settings.
confirm	Sets <i>xmail</i> to require a RETURN on commands that do not accept parameters such as the get new mail command (n). When confirm is set to ON, <i>xmail</i> only accepts RETURN and the Kill and Erase characters when you select such a command.
deletesave	Automatically deletes messages once you have saved them with the save commands.
editmail	Automatically invokes the editor when sending mail. If you turn off this option, <i>xmail</i> prompts you for header information and then for the message itself (with a > prompt for each line).
getnewmail	Automatically gets new mail when you enter your default working mailbox.
ignore_cr	Tells <i>xmail</i> to ignore the RETURN key when prompting for a command. If this option is off, RETURN acts the same as the = command: it reads the next message.
includetext	Includes the text of the message to which you are replying in your message. This option works only when the editmail option is turned on, as otherwise you are not allowed to edit the message before you add your own text.
mesg	Lets you allow or disallow the receiving of messages during the mail session.

- prompt_nl** Tells *xmail* to insert a newline in front of prompts.
- showdelete** Displays deleted messages when showing message headers.
- showlines** Tells *xmail* to display the number of lines and bytes for each message when displaying message headers. *Xmail* uses the format *lines/bytes* when displaying the size.
- showmsgs** Displays messages currently in your mailbox when you start up *xmail* or enter a new mailbox.
- strict** Does not allow lines in the header block which *xmail* does not recognize. If this option is off, you can place anything in the header block. To edit the header block you must also invoke the *editmail* option.
- verbose** Displays messages indicating *xmail*'s progress while updating a mailbox or performing other functions.
- setenv VAR value [value...]**
Sets the environment variable VAR to the value(s) given.

signfile=signaturefile
signfile=OFF

Tells *xmail* to add the text found in *signaturefile* to your message template before you reach the editor, or where to find your signature when using *sendmail*'s *put* signature command (*p*). If you specify **OFF**, *xmail* does not include a signature file.

sysalias mailname username [username...]

Creates a system alias, which is invisible (not shown by the *A* command). This is normally used by the system administrator to create a system-wide alias in the */usr/lib/xmail.rc* file. To display the system aliases, type at the *mail>* prompt:

```
o show sysaliases
```

umask value Sets the *umask* to the requested value while running *xmail*.

unalias name

Removes *name* from *xmail*'s alias list.

Defaults For Xmail Options

Xmail defaults to the following option settings:

```
maildir=.
mailbox=~/.mbox
savembox=~/.mbox
recordfile=OFF
signfile=OFF
set prompt_nl showmsgs -getnewmail -autodelete
```

```
set msg -showlines
set -ignore_cr -autocc verbose -showdelete banner
set -autoprint -confirm strict -includetext
```

Xmail Commands

When you start up *xmail*, it displays its version number and how many messages are in the mailbox you have selected or in your default mailbox if you didn't select one. *Xmail* also shows whether or not you have received new mail.

Xmail then displays its prompt. There is a set of commands which you can enter at this prompt.

Note: The term *msg#-list* refers to a list of integers which represent the message numbers. You may specify them in any of the following forms:

<i>number</i>	single message
<i>number number ...</i>	list of messages
<i>number-number</i>	range of messages
.	for current message
\$	for the last message
*	all messages in file

For example: **d 1 2 4-7 .-11 14-\$**

The *xmail* commands are:

a <i>name list</i>	Adds a user alias to <i>xmail</i> 's list.
A	Displays <i>xmail</i> 's current alias list.
c	Checks the system mailbox for new mail.
d <i>msg#-list</i>	Deletes message(s) from mailbox.
D	Deletes the current message from mailbox. This is the same as d RETURN.
e <i>msg#-list</i>	Lets you edit specified message.
E	Lets you edit the current message. This is the same as R RETURN.
f <i>msg#-list user</i>	Forwards a message to <i>user</i> .
F [-p] <i>str</i> <i>[file]</i>	Finds string in the specified file. <i>Xmail</i> looks for the files in the current mailbox. Specify the -p option if you want the output duplicated to the printer. You can specify a list of files by separating each by a space. Specify "." to indicate the current working mailbox (default).

- g [-R] *mailfile*** Goes to the specified *mailfile* after updating the current mailbox. Specify the **-R** option if you want readonly access to *mailfile*.
- G [-R] *mailfile*** Goes to the specified *mailfile* but does not update the current mailbox. Specify the **-R** option if you want readonly access to *mailfile*.
- h *msg#-list*** Shows a header for each message specified. If you specify only one number, *xmail* displays a screenful of headers starting at that message number. If you omit *msg#-list*, *xmail* displays the next screenful of messages.
- H** Shows a header for the last screenful of messages.
- l** Displays the contents of the current directory.
- m [-s '*subj*'] *user*** Sends mail to the specified *user(s)* by first invoking the editor. If you specify **-s** with a subject enclosed in quotes, *xmail* inserts this into the header block.
- n** Gets new mail from system mailbox.
- o *option*** Lets you enter an .xmailrc option from *xmail*'s prompt.
- O** Displays the current .xmailrc option settings.
- p *msg#-list*** Prints the specified message(s) on the system printer.
- P** Prints the current message on the system printer. This is the same as **p** RETURN.
- q** Exits *xmail* and updates the mailbox.
- Q** Exits *xmail*, but does not update the mailbox. Any new mail is still saved.
- r *msg#-list*** Sends a reply to specified message(s).
- R** Sends a reply to the current message. This is the same as **r** RETURN.
- s *msg#-list file*** Saves the specified message(s) to *file*. If you omit *file*, *xmail* saves the file to the default save file (usually `~/mbox`).
- S** Saves the current message to the default save file (usually `~/mbox`). This is the same as **s** RETURN.

t <i>msg#-list</i>	Types (displays) the specified message(s). You can omit “t” and simply type the message number.
T	Types (displays) the current message. This is the same as t RETURN.
u <i>msg#-list</i>	Undeletes message(s) in the mailbox.
v	Displays the version number of <i>xmail</i> .
-	Displays the previous message.
+ or =	Displays the next message.
RETURN	Displays the next message if the ignore_cr option is turned off.
!	Starts up a shell.
!command	Runs a shell command and returns to <i>xmail</i> .
.	Displays the current message/mailbox name.
?	Displays <i>xmail</i> 's help screen.

Sending Mail

After exiting the editor when sending mail, *xmail* displays a `sendmail>` prompt. At this prompt, you can enter one of the following commands:

a <i>file</i>	Add the contents of the specified file to the message.
c	Continue adding text to the message.
d	Display the message.
e	Re-edit the message.
p	Add the signature file to the message.
q	Quit (Abort).
s	Send the message.
?	Display help screen.

The following commands are only available from “`sendmail>`” if you have entered *xmail* and have read in a mailbox:

- h** Display message headers.
- r msg#-list** Read the specified messages into text.
- .** Show current message.

The /etc/usemap File

When sending mail, *xmail* looks at the address. If it is a uucp network address in the form *system!user*, *xmail* attempts to open the file */etc/usemap*. If the file exists, *xmail* tries to find a uucp path for *system*. If the path exists, *xmail* substitutes that path for *system*. If there is not an entry for *system*, *xmail* displays a warning message. If */etc/usemap* does not exist, *xmail* leaves the address alone.

Xmail expects */etc/usemap* to contain a sorted list of system names followed by at least one space, then the path to that system with a *%s* for the user name. For example:

```
flakey      doc!flakey!%s
mcp         sneezy!eagle!mcp!%s
trsvax      trsvax!%s
```

Special Features Available While Sending Mail

Xmail has several special features available when sending mail. You may send a copy of your message to a shell command or save a copy of a message from the “To:” or “Cc:” lines. You may also specify a mailing list file from there. Here is an example of how you use the features:

```
% xmail everybody
One moment please...
```

Xmail enters the editor. The following is a sample screen:

```
To: everybody *others
Subject:
Return-receipt-to: me
Cc: *yetothers |/pr | lpr/ +everybody
Bcc: theboss
```

This is a sample.

Notice the “Cc:” line. The **yetothers* tells *xmail* to open the file “yetothers” and send the message to every person in the file. The */pr | lpr/* tells *xmail* to write a copy of the message to the command “pr | lpr”. (You can use any delimiter you wish, as the character after the “|” is taken as a delimiter). The *+everybody* appends a copy of the message to the file “everybody.” So, in summary:

- * reads a file and sends mail to the list of users contained in the file.
- + appends the message to the specified file.
- | writes a copy of the message to a command.

The “Bcc:” line (blind-carbon-copy) sends a copy of the message to anyone listed, but the “Bcc” line itself is not included in the message.

The “Return-receipt-to:” line is an instruction to the receiving mailer to send mail back to the sender acknowledging the receipt of the message. In this case, an acknowledgement will be sent to the user “me”.

Also available is the “Return-receipt-from:” line. This lets you specify from whom you wish to receive a return receipt. This is useful when you are sending mail to a group of people but only want a return receipt from one or just a few.

“Return-receipt-to” and “Return-receipt-from” are mutually exclusive.

Notes

The `/usr/lib/xmail_recover` program is used to restore mail that has not been saved as a result of a system crash. This is called by `/etc/rc` when the system is booted.

Files

<code>/usr/lib/xmail</code>	mail program
<code>/usr/lib/xmail.cmd.hlp</code>	commands help text
<code>/usr/lib/xmail.opt.hlp</code>	options help text
<code>/usr/lib/xmail.set.hlp</code>	"set" help text
<code>/usr/lib/xmail_recover</code>	xmail recovery program
<code>/usr/lib/xmlock</code>	xmail system mailbox locking program
<code>/usr/spool/mail/*</code>	post office
<code>~/mbox</code>	old mail
<code>/usr/lib/xmail.rc</code>	system-wide xmail options
<code>~/xmailrc</code>	user's xmail options
<code>/tmp/xm.#</code>	xmail temporary files



Name

yes – Prints string repeatedly.

Syntax

yes [*string*]

Description

Yes repeatedly outputs “y”, or if a single string argument is given, *string* is output repeatedly. The command will continue indefinitely unless aborted. *Yes* is useful in pipes to commands that prompt for input and require a “y” response for a yes. In this case, *yes* terminates when the command that it pipes to terminates, so that no infinite loop occurs.



The following will combine the output of the parenthesized commands onto one line, which is then echoed to the end of file, log:

```
(logname; date; echo $0 $*) | xargs >>log
```

The user is asked which files in the current directory are to be printed and prints them one at a time:

```
ls | xargs -p -l lpr
```

Or many at a time:

```
ls | xargs -p -l | xargs lpr
```

The following example will execute *diff*(C) with successive pairs of arguments originally typed as shell arguments:

```
echo $* | xargs -n2 diff
```



Index - C Commands

Accounting files, printing	acctcom
Accounting, starting	accton
Archives, creating and restoring	tar
at command	at
atrm command	at
Backup, creating	backup
Backup, creating and restoring	sysadmin
Backup, dates	sddate
Backup, listing	dumpdir
Backup, restoring	restore
batch command	at
bootable XENIX system, create	hdinit
Buffers and copies a file.	tdd
C language, flow graph	cflow
C language, cross-reference	cxref
Calculator	bc
Calculator	dc
Calendar, display	cal
cancel command	lp
Cartridge tape, rewind	rewind
Cartridge tape, skip volume(s)	skip
Character translation	tr
Clear screen	clear
Clear screen	cls
Clock, sets/display	setclock
Commands, constructing and executing	xargs
Commands, execution on a remote system	remote
Commands, execution priority	nice
Commands, execution without hangups and quits	nohup
Commands, installing	install
Commands, intersystem execution	uux
Commands, options	getopt
Commands, running	runbig
Commands, scheduled execution	at
Commands, scheduled execution	cron
Communication, calling other systems	cu
Communication, copying files across systems	rcp
Communication, removing information	ipcrm
Communication, status	ipcs
Conversions, units	units
Convert and copies, file	dd
Copying files, public to private	uuto
deassign command	assign
Devices, exclusive control	assign

Devices, names	devnm
Directory, comparing	dircmp
Directory, creating	mkdir
Directory, listing	ls
Directory, listing	ll
Directory, listing	l
Directory, listing columns	lc
Directory, listing columns	lr
Directory, listing columns	lx
Directory, removing	rmdir
Directory, renaming	mvdir
Disk type	dtype
Displaying, command arguments	echo
Displaying, first lines of a file	head
Displaying, last lines of a file	tail
Displaying, line numbers	nl
Ed, restricted version	red
egrep command	grep
Environment, setting	env
Expressions, evaluating	expr
Factoring numbers	factor
fgrep command	grep
File system, backup	backup
File system, backups	sysadmin
File system, checking and repairing	fsck
File system, constructing	mkfs
File system, mount table	setmnt
File system, mounting	mount
File system, names from inode numbers	ncheck
File system, ownership	quot
File system, unmounting	umount
File, access and modification dates	settime
File, access and modification times	touch
File, access permissions	chmod
File, building special files	mknod
File, checksum and blocks	sum
File, comparing	cmp
File, comparing	bdiff
File, comparing side-by-side	sdiff
File, comparing text	diff
File, comparing text	diff3
File, compressing and expanding	pack
File, concatenating and displaying	cat
File, converting and copying	dd
File, copying	cp
File, copying archives	cpio
File, copying groups	copy
File, counting lines, words and characters	wc

File, creation mode mask	umask
File, displaying	pr
File, displaying repeated lines	uniq
File, group ID	chgrp
File, hexadecimal display	hd
File, identifying	what
File, intersystem copy	uucp
File, linking	ln
File, locating	find
File, moving and renaming	mv
File, octal display	od
File, owner ID	chown
File, perusing	pg
File, printing	lpr
File, removing	rm
File, scanning	bfs
File, selecting common lines	comm
File, sorting	sort
File, splitting by context	csplit
File, splitting by lines	split
File, type	file
File, viewing	more
Formats media	format
Function keys, assigning	setkey
Group file	grpcheck
Group, switching	newgrp
Inodes, clearing	clri
International keyboards, activate	keyboard
Large letters	banner
Line, reading from input	line
Lines, finding in a sorted list	look
Login, name	logname
Lpr command	lp
Mail	mail
Manual page, locating	help
Micnet, creating and operating	netutil
MS-DOS, accessing files	dos
News	news
Password, aging	pwadmin
Password, changing	passwd
Password, file check	pwcheck
Pathname, directory name	dirname
Pathnames, filename	basename
Pattern, searching	grep
Pattern, searching and processing	awk
pcat command	pack
Pipe, creating a tee	tee
Print requests, enabling and disabling	accept

Printing, adding printers	lpinit
Printing, configuring the system	lpadmin
Printing, moving requests	lpsched
Printing, starting the system	lpsched
Printing, status	lpstat
Printing, stopping the system	lpsched
Process, status	ps
Process, temporary suspension	sleep
Process, terminating	kill
Process, waiting for background process	wait
Random number	random
Relations, joining	join
Reminder service	calendar
Return value, nonzero	false
Return value, repeated string	yes
Return value, zero	true
Root directory	chroot
Shell	sh
Shell, restricted	rsh
Shell, visual	vsh
Single user mode, accessing	sulogin
System, access	login
System, current name	uname
System, disk usage	du
System, free disk blocks	df
System, information	pstat
System, message display	dmesg
System, stopping	haltsys
System, stopping	shutdown
System, super-block	sync
Terminal, disable login	disable
Terminal, enabling logins	enable
Terminal, enabling messages	mesg
Terminal, name	tty
Terminal, setting modes	stty
Terminal, setting modes	tset
Terminal, writing to all	wall
Testing conditions	test
Text editor, line	ed
Text editor, line	ex
Text editor, screen	vi
Text editor, stream	sed
Text files, format	newform
Time of day	asktime
unpack command	pack
User, adding to the system	mkuser
User, listing	who
User, listing action	whodo

User, removing from the system	rmuser
User, switching	su
User, writing to a terminal	write
Users, information	finger
Uucp directory cleanup	uuclean
Uucp files, administering	uuinstall
Uucp network monitoring	uusub
Uucp status inquiry	uustat
Uulog command	uucp
Uuname command	uucp
Working directory	cd
Working directory	pwd
Write another user	hello
IDs, user and group	id



XENIX[®] System V/286

Operating System

Commands(M)

Information in this document is subject to change without notice and does not represent a commitment on the part of Microsoft Corporation. The software described in this document is furnished under a license agreement or nondisclosure agreement. The software may be used or copied only in accordance with the terms of the agreement. It is against the law to copy this software on magnetic tape, disk, or any other medium for any purpose other than the purchaser's personal use.

© Copyright Microsoft Corporation, 1985

Microsoft, the Microsoft logo, and XENIX are registered trademarks, and The High Performance Software is a trademark of Microsoft Corporation.

UNIX is a trademark of AT&T.

Document Number: 290190009-500-R00-0785